
UNIDAD 06

Funciones

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

- 01 Empaquetar código con un **nombre**
- 02 **Parámetros** y `return`
- 03 Por qué las variables de adentro **no se ven** afuera
- 04 **JSDoc**: documentar como se pide en el curso
- 05 `Math.random()` y `rndInt(min, max)`

La clase pasada te quedaron programas largos y repetitivos. **Hoy se paga esa promesa.**

PARTE 1

| El problema

La misma cuenta, tres veces

src/app.js

```
const deviation1 = (measured1 - 380) / 380 * 100
console.log(`Tablero 1: ${deviation1.toFixed(2)} %`)

const deviation2 = (measured2 - 380) / 380 * 100
console.log(`Tablero 2: ${deviation2.toFixed(2)} %`)

const deviation3 = (measured3 - 380) / 380 * 100
console.log(`Tablero 3: ${deviation3.toFixed(2)} %`)
```

El 380 está escrito tres veces. La fórmula también.

PREGUNTA

**Si mañana cambia la fórmula,
¿cuántos lugares tocás?**

¿Y cuántas probabilidades hay de que te olvides de uno?

PREGUNTA

Esto ya lo conocen:
 $f(x) = 2x + 1$

¿Qué es una función, en matemática?

La misma idea, otra notación

ANÁLISIS

$$f(x) = 2x + 1$$

Una **regla** que a cada valor del **dominio** le asigna uno de la **imagen**.

JAVASCRIPT

```
const f = (x) => 2 * x + 1
```

El mismo concepto, con paréntesis y una flecha.

No estamos aprendiendo algo nuevo: estamos **escribiéndolo distinto**.

Pieza por pieza

EN MATEMÁTICA

f , el nombre

x , la variable independiente

la regla $2x + 1$

evaluar $f(3)$

el valor $f(3) = 7$

el **dominio**

EN PROGRAMACIÓN

el nombre de la función

el **parámetro**

el **cuerpo**

llamar `f(3)`

lo que entrega el `return`

los valores que tiene sentido pasarle

Y las de varias variables también: $P(V, I) = V \cdot I$ es `power(voltage, current)`.

Definir no es evaluar

Escribir $f(x) = 2x + 1$ en el pizarrón
no calcula nada

Es una **definición**. Recién cuando escribís $f(3)$ se hace la cuenta.

En programación es **idéntico**: declarar una función
no la ejecuta. Recién al **llamarla** corre el cuerpo.

Dónde se rompe la analogía

- 01 Puede **no devolver nada** — `printHeader()` sólo imprime.
- 02 Puede **tener efectos** — `console.log` cambia la pantalla.
- 03 Puede devolver algo **distinto cada vez**.

ERROR FRECUENTE

Hoy vas a escribir `rndInt(1, 6)`, que con los mismos argumentos devuelve valores distintos. **Eso no es una función matemática:** la definición exige un solo valor por cada elemento del dominio.

Funciones puras

Las que **sí** cumplen: mismos argumentos, mismo resultado, sin efectos.

```
src/app.js
```

```
power(380, 4.2) // 1596, siempre
```

Son las más fáciles de probar y de reutilizar.
Cuando puedas elegir, escribí funciones puras:

DEJÁ EL `console.log` Y EL AZAR EN EL PROGRAMA PRINCIPAL.

PARTE 2

| Declarar una función

Una sola vez

src/app.js

```
const deviation = (measured, nominal) => {  
  return (measured - nominal) / nominal * 100  
}  
  
console.log(deviation(372.5, 380).toFixed(2))
```

SALIDA

```
-1.97
```

Declararla **no ejecuta nada**. Es escribir la receta. Recién al **llamarla** se ejecuta el cuerpo.

Cómo se lee

src/app.js

```
const deviation = (measured, nominal) => {  
  //   ↑           ↑           ↑  
  //   |           |           | el cuerpo, entre llaves  
  //   |           | parámetros: los datos que necesita  
  //   | el nombre con el que la vas a llamar  
}
```

La forma corta

Si el cuerpo es **una sola expresión que se devuelve**, se omiten las llaves y el `return`.

```
src/app.js
```

```
const isEven = (n) => n % 2 === 0  
const toFahrenheit = (celsius) => celsius * 9 / 5 + 32
```

Si entra cómoda en una línea, la forma corta. Si hay más de una instrucción, llaves y `return`.
La legibilidad manda.

Y la forma que NO usamos

× TRADICIONAL

```
function deviation (m, n) {  
  return (m - n) / n * 100  
}
```

La vas a ver en tutoriales y en código viejo.

✓ ARROW

```
const deviation = (m, n) => {  
  return (m - n) / n * 100  
}
```

Es la de la cátedra.

Hacen exactamente lo mismo. La tradicional la ves para **reconocerla**, no para usarla.

PARTE 3

| Parámetros y return

Dos palabras parecidas

PARÁMETRO

El nombre en la **declaración**. Un hueco a llenar.

```
const power =  
  (voltage, current) => ...
```

ARGUMENTO

El **valor concreto** que pasás al llamarla.

```
power(380, 4.2)
```

ERROR FRECUENTE

El orden importa. `deviation(380, 372.5)` no es lo mismo que `deviation(372.5, 380)` : da `2.01 %` en vez de `-1.97 %`, sin ningún error.

Devuelve y termina

src/app.js

```
const classify = (measured) => {  
  if (measured < 361) {  
    return 'BAJA'  
  }  
  if (measured <= 399) {  
    return 'NORMAL'  
  }  
  return 'ALTA'  
}
```

No hacen falta los `else`: como el `return` **corta**, si llegaste a la segunda pregunta es porque la primera dio `false`.

Lo que devuelve, hay que usarlo

src/app.js

```
deviation(372.5, 380)           // se calcula y se tira  
  
const result = deviation(372.5, 380) // se guarda  
  
console.log(deviation(372.5, 380)) // se usa directo
```

La primera línea no da error. Simplemente hace la cuenta y la descarta.

PREGUNTA

¿Qué devuelve una función que no tiene `return`?

```
const x = printHeader('TAB-3')
```

Funciones que sólo hacen

src/app.js

```
const printHeader = (panelId) => {  
  console.log('_____')  
  console.log(`Tablero ${panelId}`)  
  console.log('_____')  
}
```

Está perfecto que no devuelvan nada. Pero su resultado es `undefined`:
no intentes usarlo.

ERROR FRECUENTE

Qué se puede devolver: un valor primitivo — número, texto, booleano — y desde la unidad 07, un vector. **Objetos y JSON no.** Por ahora, si necesitás devolver tres cosas, son tres funciones; desde la unidad 07 pueden viajar juntas en un vector.

Lo de adentro no se ve

src/app.js

```
const power = (voltage, current) => {  
  const result = voltage * current  
  return result  
}  
  
console.log(result)
```

SALIDA

```
ReferenceError: result is not defined
```

Y eso es **bueno**: podés usar `result`, `i` o `total` adentro sin miedo a pisar nada de afuera.

La regla

Una función trabaja con sus parámetros

Si necesita un dato, **se lo pasás**. No lo lee de afuera.

Una función que lee variables del programa principal deja de ser reutilizable: sólo sirve en ese programa, con esos nombres.

PARTE 4

JSDoc

Toda función lleva uno

src/app.js

```
/**
 * Calcula la desviación porcentual respecto del valor nominal.
 * @param {number} measured - Valor medido
 * @param {number} nominal - Valor nominal de referencia
 * @returns {number} Desviación en porcentaje
 */
const deviation = (measured, nominal) => (measured - nominal) / nominal * 100
```

Primera línea: **qué hace**. Un `@param` por parámetro. Un `@returns` . **Se corrige**.

Y se paga solo

En VSCode, cuando escribís `deviation(` te muestra la descripción y los parámetros en un cartelito.

NOMBRES

`isEven`, `hasAlarm` para las que devuelven **booleano**.

VERBOS

`calculateAverage`, `convertTemperature` para las que **calculan**.

Si el nombre de una función necesita un «y», son dos funciones.

PARTE 5

Números aleatorios

Math.random()

SALIDA

```
0.7339461987959066  
0.0704937964111689  
0.8267381760766475
```

Decimales entre **0 y 1**. Incluye el 0, **nunca llega al 1**.

Así solo no sirve de mucho: nadie mide tensiones entre 0 y 1. Hay que **estirlo**.

PREGUNTA

¿Cómo saco un número entero del 1 al 6?

Tenés `Math.random()` , que da decimales entre 0 y 1.

Tres pasos

01 **Estirar** — `Math.random() * (max - min + 1)` → decimal de 0 a 6

02 **Recortar** — `Math.floor(...)` → entero de 0 a 5

03 **Correr** — `+ min` → entero de 1 a 6

¿Por qué el `+ 1` ? Porque los extremos cuentan los dos. Del 1 al 6 hay **seis** números, no cinco: `6 - 1 = 5`, y le sumás uno.

La función del curso

src/app.js

```
/**
 * Genera un número entero aleatorio en el rango [min, max].
 * @param {number} min - Valor mínimo del rango (inclusive)
 * @param {number} max - Valor máximo del rango (inclusive)
 * @returns {number} Entero aleatorio entre min y max
 */
const rndInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min
```

Copiala **tal cual**. La vas a usar en todos los TP y en los parciales, siempre con este nombre y esta firma.

Sin el + 1, el máximo no sale nunca

SALIDA

```
En 1000 tiradas de rndInt(1, 6), ¿cuántos 6 salieron?
```

```
con el + 1: 177
```

```
sin el + 1: 0
```

ERROR FRECUENTE

El programa **anda**, los números parecen razonables, y falta uno solo de todos los posibles. Es de los errores más difíciles de notar.

PARTE 6

| El pago de la promesa

Las piezas

src/app.js

```
const rndInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min

const simulateMeasurement = () => rndInt(355, 405)

const deviation = (measured) =>
  (measured - NOMINAL_VOLTAGE) / NOMINAL_VOLTAGE * 100

const isWithinRange = (measured) =>
  measured >= NOMINAL_VOLTAGE * (1 - TOLERANCE) &&
  measured <= NOMINAL_VOLTAGE * (1 + TOLERANCE)
```

Cada una hace **una sola cosa**. El JSDoc va en el archivo; acá se omite por espacio.

Y el programa principal

src/app.js

```
for (let i = 1; i <= 8; i++) {  
  printLine(i, simulateMeasurement())  
}
```

Dos líneas

Cada función cuenta **cómo**. El programa principal cuenta **qué**.

Y si mañana cambia la tolerancia, hay **un solo lugar** donde tocar.

PARTE 7

| Ahora ustedes

Cómo trabajamos

- 01 Respetá **la firma exacta** que pide cada consigna. Los nombres son los del curso.
- 02 **JSDoc en todas.** Sin excepción.
- 03 Escribí la función **y** el `console.log` que la prueba.
- 04 Probá con **casos límite**: el mínimo, el máximo, el 1, el 0.
- 05 Una función que anda se deja quieta: la siguiente **la va a usar**.

A1 • Un decimal aleatorio

Un sensor de temperatura no devuelve enteros. Escribí `rndFloat(min, max)`, que dé un **decimal** aleatorio dentro del rango.

src/app.js

```
const rndFloat = (min, max) => // ...  
  
console.log(rndFloat(20, 30).toFixed(2))
```

SALIDA

24.71

Arrancá de `rndInt` y **sacale lo que sobra**.

A2 • Par o impar, a elección

`rndParity(min, max, wantEven)` devuelve un entero del rango: **par** si `wantEven` es `true`, **impar** si es `false`.

src/app.js

```
const parity = prompt('¿Par o impar? ')\n\nconsole.log(rndParity(10, 20, parity === 'par'))
```

SALIDA

```
¿Par o impar? par\n16
```

Es la primera función del curso con un **parámetro booleano**. Mirá cómo lo arma el programa principal.

PREGUNTA

¿Cómo hace un programa para saber si un número es primo?

Primo: divisible sólo por **1** y por **sí mismo**. El 1 no es primo.

B1 • isPrime(n)

Devuelve `true` si `n` es primo y `false` si no. **Es la pieza:** la consigna que sigue se apoya en ella.

src/app.js

```
const isPrime = (n) => // ...  
  
console.log(isPrime(7))    // true  
console.log(isPrime(9))    // false  
console.log(isPrime(1))    // false
```

Un bucle que prueba divisores. En cuanto encontrás uno, `return false` — y el bucle **se corta solo**.

B2 • Un primo al azar

`rndPrime(min, max)` devuelve un entero **primo** aleatorio del rango. No la escribas de cero: ya tenés las dos piezas.

src/app.js

```
const rndPrime = (min, max) => // ...  
  
console.log(rndPrime(1, 50))
```

SALIDA

31

ERROR FRECUENTE

¿Qué pasa si en el rango **no hay ningún primo**? Probá `rndPrime(24, 28)`.

Lo que acaban de construir

01 `rndParity` usa `rndInt`

02 `rndPrime` usa `rndInt` **y** `isPrime`

Mirá `rndPrime`: no tiene **una sola línea** de primos ni de sorteo.

LLAMA A DOS FUNCIONES QUE YA ANDABAN. ESO ES COMPONER, Y ES DE LO QUE ESTÁN HECHOS LOS PROGRAMAS GRANDES.

Lo que hay que llevarse

- 01 Una función = **un nombre + una tarea**.
- 02 Trabaja con **sus parámetros**. No lee de afuera.
- 03 `return` devuelve **y corta**.
- 04 **JSDoc siempre**. Se corrige.
- 05 `rndInt(min, max)` de memoria — y el `+ 1`.
- 06 Una función puede **usar otra**. Así se arman los programas grandes.

PRÓXIMA CLASE • UNIDAD 07

Vectores

Por fin un lugar donde **guardar** las mediciones

Se generan con `rndInt()` · se recorren con los bucles de la 05 · y cada cálculo es una función